SWINBURNE UNIVERSITY OF TECHNOLOGY

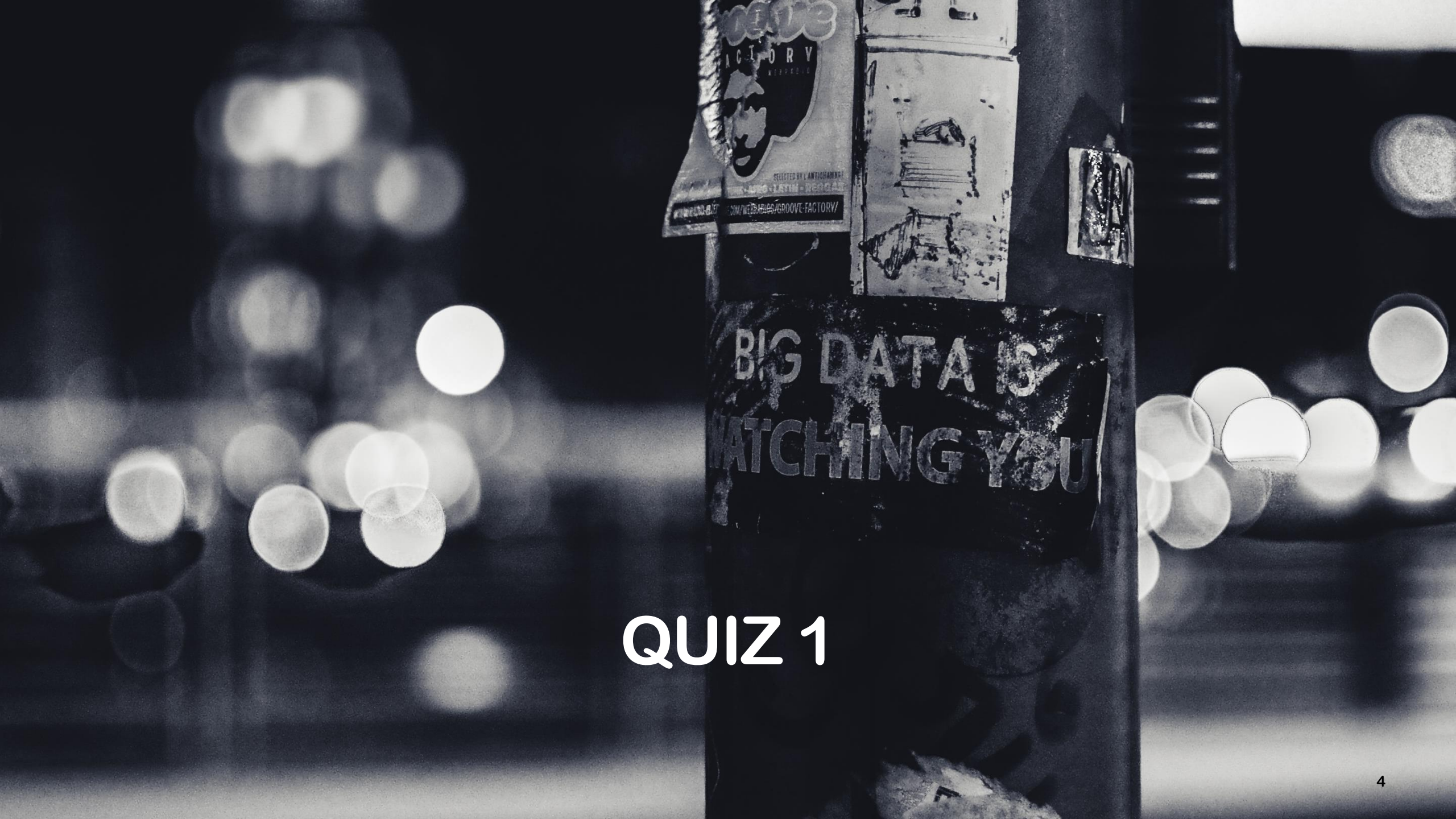
WEEK4 - COS20028
(BIG DATA ARCHITECTURE AND APPLICATION)

Dr Pei-Wei Tsai (Unit Convenor)

Yicun Tian(Lecturer, Tutor)

Week 4 Overview: What You'll Learn

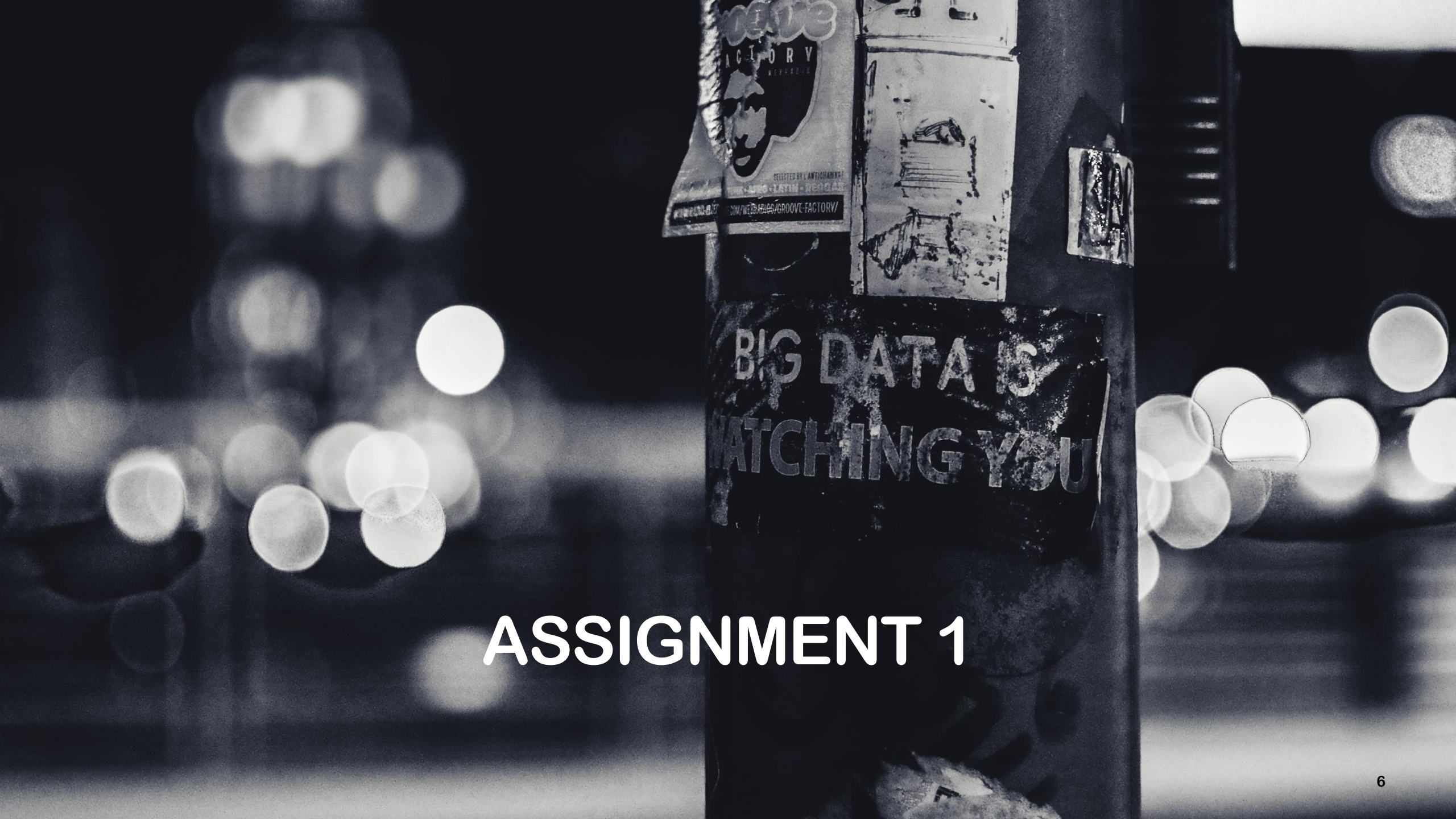
Source	Content
Online Videos (Canvas)	Part 1: Going deeper into Hadoop API Part 2: Setting Up and Tearing Down Mappers and Reducers
Live Lecture	Quiz 1 Assignment 1 Recap ToolRunner & Combiner
Lab Activities	Test 1 ToolRunner Combiner



QUIZ 1

Quiz

- Duration: 1 hour
- Number of questions: 30
- Theory questions 60 % + Application questions: 40%
- Question types: Multiple-choice + Multiple-answer + Fill in the blanks
- Content coverage: Week1-3 online learning + lecturer + lab
- Students must have their devices ready
- Students must bring their student card to attend their registered lab session



ASSIGNMENT 1

RECAP – TOOLRUNNER

Open-ended Question

Question:

In a Hadoop MapReduce job, suppose you want to make the program flexible so that users can control case sensitivity during execution.

1. **modifying the source code directly**

```
boolean caseSensitive = true;  // or false
```

2. prefer to pass parameters dynamically through the command line

```
hadoop jar myjob.jar stubs.WordCount -D caseSensitive=true input output
```

ToolRunner– T/F

1) ToolRunner is mandatory for all MapReduce driver classes.
2) ToolRunner internally uses the GenericOptionsParser class.
3) One of the purposes of ToolRunner is to specify configuration options for a MapReduce job.
4) ToolRunner cannot be used to set items for the Distributed Cache.
5) If you don’t use ToolRunner, you must handle command-line arguments and configurations manually.

ToolRunner

- **Purpose of ToolRunner**
  - **Simplifies running MapReduce driver classes.**
  - **Handles configuration and command-line arguments.**
- **Not Mandatory but Recommended**
  - **You can write MapReduce drivers without ToolRunner, but it is a best practice.**
- **GenericOptionsParser**
  - **ToolRunner uses this class internally to parse Hadoop command-line options.**
- **Configuration Options**
  - **ToolRunner makes it easier to pass custom configuration values into your job.**
- **Distributed Cache Support**
  - **ToolRunner lets you specify items for Hadoop's Distributed Cache, allowing files to be shared across all nodes.**

ToolRunner– T/F

1) ToolRunner is mandatory for all MapReduce driver classes. ❌ False Explanation: Using ToolRunner is not mandatory, but it is considered a best practice for managing configurations and command-line options.
2) ToolRunner internally uses the GenericOptionsParser class. ✅ True
3) One of the purposes of ToolRunner is to specify configuration options for a MapReduce job. ✅ True
4) ToolRunner cannot be used to set items for the Distributed Cache. ❌ False Explanation: ToolRunner can specify items for the Distributed Cache, making data distribution across nodes simpler.
5) If you don't use ToolRunner, you must handle command-line arguments and configurations manually.Answer: ✅ True

With and without ToolRunner in Driver Class

Change	Added/Modified Content	Purpose
Class Definition	extends Configured implements Tool	Enables the class to support ToolRunner
main Method	Use ToolRunner.run()	Provides a unified entry point and automatically parses configuration and arguments
New Method	run(String[] args)	Core execution logic of the MapReduce Job
Get Configuration	getConf()	Automatically uses the configuration provided by ToolRunner

Import Configured

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
```

Aspect	Configuration	Configured
Package	org.apache.hadoop.conf.Configuration	org.apache.hadoop.conf.Configured
Type	**Class**	**Abstract class**
Purpose	Stores **key-value pairs** for Hadoop's configuration, such as file paths, I/O settings, cluster info, etc.	Provides a **base class** for Hadoop tools that need access to a Configuration object.
Usage	Used whenever you want to **create or modify** Hadoop settings directly.	Usually extended by custom classes when you want them to **automatically get and use** a Hadoop configuration.
How to Use	Create an instance directly: Configuration conf = new Configuration();	Extend it in your class: public class MyTool extends Configured implements Tool { ... }
Common Scenario	Reading/writing configuration properties for HDFS, MapReduce, etc.	Used with Tool interface to run Hadoop jobs with ToolRunner.
Example	Configuration conf = new Configuration();	public int run(String[] args) { Configuration conf = getConf();}

Import Tool

```
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;
```

Package	What It Provides	Why We Need It
org.apache.hadoop.util.Tool	Defines a **standard interface** for driver classes	Allows us to implement the Tool interface and manage configurations consistently
org.apache.hadoop.util.ToolRunner	A **helper class** to execute Tool-based driver classes	Simplifies running MapReduce jobs and reduces boilerplate code

Class Definition

```
public class WordCount extends Configured implements Tool
```

1) **public class WordCount** : Think of this class as the controller of the entire MapReduce program.
2) **extends Configured**:
   - Configured is a Hadoop-provided base class.
   - It manages the Configuration object, which contains all the job's settings.
3) **implements Tool**: Implementing Tool makes our driver class compatible with ToolRunner.

Component	Role	What It Does
Configured	Base class	Manages the Hadoop configuration object
Tool	Interface	Standardizes how to write driver classes
ToolRunner	Helper class	Runs Tool-based classes and automatically sets up the environment

Use ToolRunner.run()

```
public static void main(String[] args) throws Exception {
    int exitCode = ToolRunner.run(new Configuration(), new WordCount(), args);
    System.exit(exitCode);
}
```

- **Method Signature**
  - **Command : hadoop jar WordCount.jar input output**
- **ToolRunner.run(…)**

Parameter	What It Is	Purpose
new Configuration()	Hadoop **Configuration** object	Loads Hadoop's default settings and prepares job-level configurations.
new WordCount()	Our **driver class** instance	This is the class where run() is defined and the job is configured.
args	Command-line arguments	Passes input/output paths and any other job-specific parameters.

Use ToolRunner.run()

```
public static void main(String[] args) throws Exception {
    int exitCode = ToolRunner.run(new Configuration(), new WordCount(), args);
    System.exit(exitCode);
}
```

- Start program → main() is called.
- ToolRunner:
  - Sets up the environment.
  - Parses arguments.
  - Calls WordCount.run().
- Inside run():
  - Job configuration is set.
  - Job is submitted to Hadoop.
- Job completes → returns an exit code.
- System.exit(exitCode) ends the program.

run(String[] args)

```
public int run(String[] args) throws Exception {
    if (args.length != 2) {
        System.out.printf("Usage: WordCount <input dir> <output dir>\n", getClass().getName());
        return -1;
    }

    Configuration conf = getConf();
    Job job = Job.getInstance(conf);
    job.setJobName("Word Count");
    job.setJarByClass(WordCount.class);

    FileInputFormat.setInputPaths(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

stubs.WordCount

Get Configuration

```
Configuration conf = getConf();
Job job = Job.getInstance(conf);
job.setJobName("Word Count");
job.setJarByClass(WordCount.class);
```

- getConf() comes from Configured (our parent class).
- Because we called ToolRunner.run() in main(), ToolRunner has already:
  - Created the configuration.
  - Merged any command-line parameters (e.g. -D mapreduce.job.reduces=2).
- Using getConf() ensures we access the same configuration prepared by ToolRunner.
- If we used new Configuration() here, we would lose any parameters passed by ToolRunner.

1) Job.getInstance(conf): Creates a Hadoop job instance using the configuration prepared by ToolRunner.
2) setJobName(): Gives the job a meaningful name for tracking in the Hadoop console.
3) setJarByClass(): Ensures that Hadoop knows where to find our driver class.

Driver– T/F

1) ToolRunner.run() is used to automatically create and pass a Configuration object to the run() method.
2) If we create a new Configuration() object inside the run() method instead of using getConf(), we might lose the parameters passed via -D on the command line.
3) getClass().getName() is used to retrieve the simple class name only, without the package name.
4) Job.getInstance(conf) is responsible for creating a new Hadoop job instance using the given configuration prepared by ToolRunner.
5) Using setJarByClass(WordCount.class) tells Hadoop where to find the driver class when running the job.

Driver– T/F

1) ToolRunner.run() is used to automatically create and pass a Configuration object to the run() method. ✅ True
2) If we create a new Configuration() object inside the run() method instead of using getConf(), we might lose the parameters passed via -D on the command line. ✅ True
3) getClass().getName() is used to retrieve the simple class name only, without the package name. ❌ False (It returns the full class name, including the package.)
4) Job.getInstance(conf) is responsible for creating a new Hadoop job instance using the given configuration prepared by ToolRunner. ✅ True
5) Using setJarByClass(WordCount.class) tells Hadoop where to find the driver class when running the job. ✅ True

Mapper

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWrita

    boolean caseSensitive;

    @Override
    public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
        String line = value.toString();

        Configuration conf = context.getConfiguration();
        caseSensitive = conf.getBoolean("caseSensitive", false);

        if (!caseSensitive) {
            line = line.toLowerCase();
        }

        for (String word : line.split("\\W+")) {
            if (word.length() > 0) {
                context.write(new Text(word), new IntWritable(1));
```

hadoop jar
toolrunner.jar
stubs.WordCount
-D caseSensitive=true
shakespeare
toolrunneroutput

If -D caseSensitive=true:

- **No lowercase conversion**
- **"Apple" and "apple" are treated as different words.**

If -D caseSensitive=false (default):

- **Converts all words to lowercase**
- **"Apple", apple, and APPLE are treated as the same word.**

Mapper– T/F

1) The Configuration conf = context.getConfiguration() retrieves the job's configuration settings, allowing the mapper to access parameters passed from the driver.
2) If the configuration parameter caseSensitive is not explicitly set, the mapper will treat all input as case-insensitive by default.
3) The line line = line.toLowerCase() will only execute when caseSensitive is set to true.
4) The caseSensitive variable gets its value directly from the Hadoop Configuration object, meaning its behavior can be controlled by setting the parameter "caseSensitive" in the job's configuration.

Mapper– T/F

1) The Configuration conf = context.getConfiguration() retrieves the job's configuration settings, allowing the mapper to access parameters passed from the driver. ✅ True
2) If the configuration parameter caseSensitive is not explicitly set, the mapper will treat all input as case-insensitive by default. ✅ True (default value is false)
3) The line line = line.toLowerCase() will only execute when caseSensitive is set to true.

   ❌ False(It executes when caseSensitive is false, meaning we want case-insensitive matching.)
4) The caseSensitive variable gets its value directly from the Hadoop Configuration object, meaning its behavior can be controlled by setting the parameter "caseSensitive" in the job's configuration. ✅ True

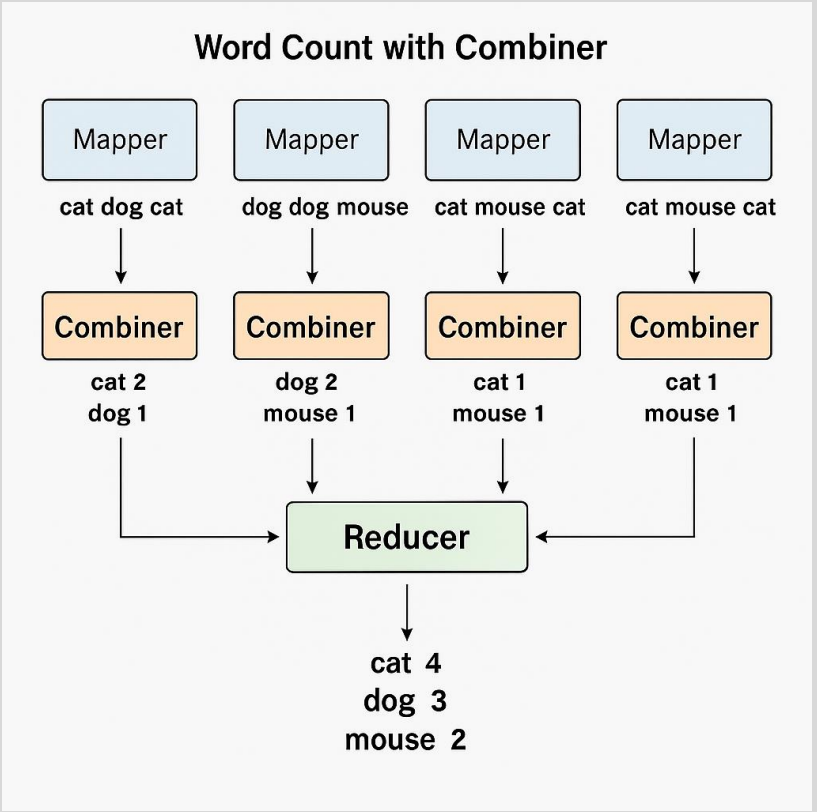
RECAP – COMBINERS

COMBINERS – T/F

1) A Combiner acts as a mini-reducer that processes Mapper output locally before sending it to Reducers.
2) One of the main purposes of a Combiner is to reduce network traffic between Mappers and Reducers.
3) Without using a Combiner, all intermediate data produced by the Mapper is sent directly to the Reducers.
4) The Combiner always produces the same final results as the Reducer, regardless of the type of operation.
5) The Combiner can have different input and output data types compared to the Reducer.
6) Combiner and Reducer always use the same code implementation in Hadoop.
7) If the Combiner is used, the number of records transferred across the network will usually decrease.

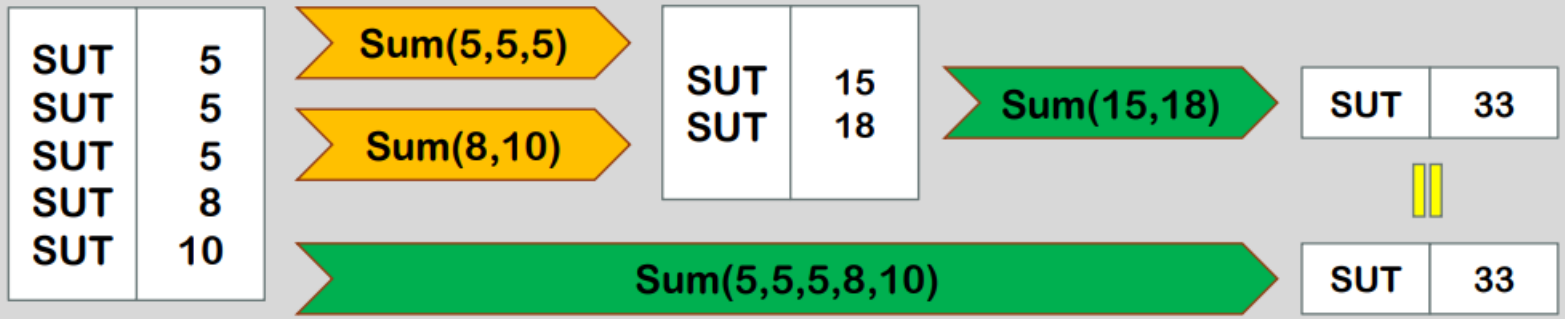
COMBINERS

- The Problem
  - Mapper usually produces a large amount of intermediate data.
  - All this data must be sent to Reducers.
  - This causes high network traffic.
- The Solution: Combiner
  - A Combiner is like a mini-reducer.
  - It runs locally on the Mapper's output.
  - It reduces the volume of intermediate data before sending it to the Reducers.
  - Final output from Combiner is then sent to Reducers.
- Combiner vs Reducer
  - Combiner often uses the same code as the Reducer.
  - This works only if the operation is:
    - Commutative → order of operations doesn't matter.
    - Associative → grouping doesn't change results.
  - Input and output data types for the Combiner must match those of the Reducer

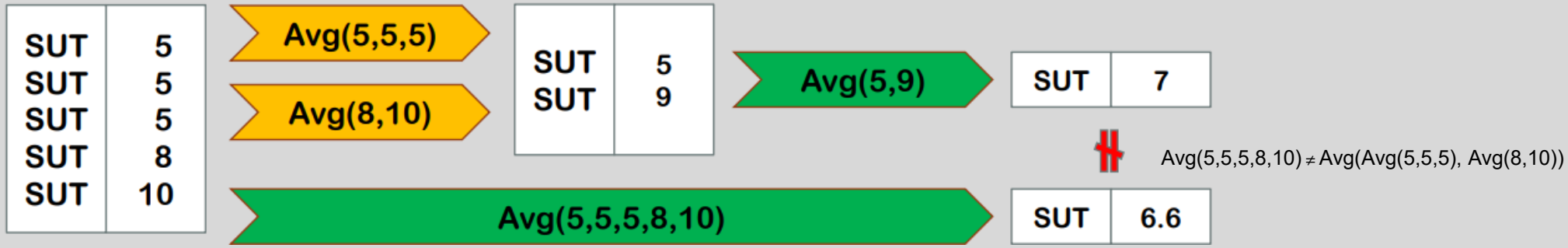


Combiners and Reducers

- If the operation is associative and commutative. (Ex: SumReducer)



- If the operation is not associative and commutative. (Ex: AverageReducer)



COMBINERS

1) A Combiner acts as a mini-reducer that processes Mapper output locally before sending it to Reducers. ✅ True
2) One of the main purposes of a Combiner is to reduce network traffic between Mappers and Reducers. ✅ True
3) Without using a Combiner, all intermediate data produced by the Mapper is sent directly to the Reducers. ✅ True
4) The Combiner always produces the same final results as the Reducer, regardless of the type of operation.
   ❌ False *(Only works when the operation is commutative and associative.)*
5) The Combiner can have different input and output data types compared to the Reducer.
   ❌ False *(They must have identical input and output data types.)*
6) Combiner and Reducer always use the same code implementation in Hadoop.
   ❌ False *(They can use the same code, but it's not required.)*
7) If the Combiner is used, the number of records transferred across the network will usually decrease. ✅ True

FileSystem

```
Configuration conf = new Configuration();
FileSystem fs = FileSystem.get(conf);
Path path = new Path("/user/hadoop/input.txt");
FSDataInputStream inputStream = fs.open(path);
```

1) Why Programmatic Access?
   - Move beyond command-line → access HDFS inside code
   - Read/write side data in MapReduce
   - Allow external programs to read MapReduce results
2) HDFS Is NOT a General-Purpose Filesystem
   - Write-once, read-many → files cannot be modified
   - Optimized for large sequential I/O, not random updates
3) Hadoop's FileSystem API
   - Provides unified interface to multiple file systems:
     - HDFS / Local FS / Amazon S3
   - Use Java APIs to open, read, and write files